Labyrinthe au trésor (édition Junior) : guide de l'enseignant

En bref

Âge	8 à 9 ans, avec l'édition **Junior** de PyGameMaker
Durée	90 minutes. C'est la deuxième leçon de labyrinthe : fais d'abord « Mon premier labyrinthe » si tu peux
Les élèves créent	Un labyrinthe au trésor : dessiner le héros, créer les objets, construire un labyrinthe avec des portes, des gemmes, une clé, un cœur et des bombes, le programmer, l'exporter en un seul fichier .html
Il faut	PyGameMaker en mode Junior, le projet de départ, un navigateur web. Internet n'est **pas** nécessaire pour jouer à ce jeu
Fiche de l'élève	Labyrinthe au trésor Junior : fiche de l'élève (une par élève)

Toute la leçon utilise uniquement des blocs qui existent dans la boîte à outils Junior. Le jeu terminé a été lancé dans le vrai moteur de jeu, exporté, puis joué dans un navigateur pour vérifier chaque mécanisme.

> cette leçon est ambitieuse pour des élèves de 8 à 9 ans en 90 minutes. Si ta classe est lente, arrête-toi après 5b (les gemmes) et termine à la séance suivante. Les priorités sont indiquées dans le plan de la leçon.

Avant la leçon (15 minutes, une seule fois)

1. **Active le mode Junior** sur chaque ordinateur : Outils > Préférences > Général > Édition de l'IDE > Junior, puis redémarre l'IDE.
2. **Copie le projet de départ** (Junior-Treasure-Starter.zip, sur la page du wiki ou dans wiki/downloads/) et décompresse-le pour que chaque élève ait un dossier appelé **Treasure Maze**.
3. **Essaie la leçon toi-même une fois.** La version terminée se trouve dans Junior-Treasure-Finished.zip, pour comparer.
4. **Utilise un PyGameMaker à jour pour l'export.** Les anciennes versions de l'export HTML5 ne géraient pas « Détruire toutes les instances de » : la clé disparaissait sans ouvrir les portes dans le navigateur. Cela a été corrigé en même temps que cette leçon.
5. **Décide comment le jeu arrivera sur un téléphone** (voir « Jouer sur un téléphone »).
6. Imprime la fiche de l'élève, ou affiche-la sur un deuxième écran.

Ce que contient le projet de départ

- **Sept images prêtes** : spr_wall, spr_door, spr_gem, spr_key, spr_heart, spr_bomb, spr_goal.
- **obj_wall**, avec spr_wall, **pas encore Solide**. Il doit exister parce que le bord de la salle est construit avec. Les élèves cochent Solide dans la partie 3.
- **room0** : 960 sur 640 pixels (30 sur 20 carrés), sur une grille de 32 pixels, avec le bord construit et rien à l'intérieur.

Les élèves dessinent seulement le héros. Tout le reste, ils le créent, le construisent et le programment.

Taille de la fenêtre (si un élève veut une zone de jeu plus grande)

La fenêtre du jeu a exactement la taille de la salle. « Mon premier labyrinthe » utilise une petite salle (640 sur 416 pixels, soit 20 sur 13 carrés) ; cette leçon en utilise une plus grande (960 sur 640 pixels, soit 30 sur 20 carrés), qui laisse la place à un labyrinthe plus long, à des couloirs plus larges et à plus d'objets. Les élèves ne règlent jamais la taille de la fenêtre eux-mêmes, et la fiche de l'élève n'en parle pas.

Si un élève demande une fenêtre plus grande :

- **Réponse la plus simple :** la salle plus grande est déjà celle de ce projet de départ. Un élève qui a terminé « Mon premier labyrinthe » et le trouve à l'étroit peut simplement faire cette leçon ensuite.
- **N'étire pas simplement une salle existante.** Les murs du contour sont placés carré par carré : dans une salle agrandie, un bord serait ouvert et il faudrait d'abord le reconstruire, sinon le héros sortirait du labyrinthe par là.
- **Tous les chemins doivent rester larges de deux carrés**, quelle que soit la taille de la salle.
- **Sur un téléphone**, la page exportée se réduit pour tenir dans la largeur de l'écran : une salle plus grande donne donc une image plus petite. La page a été vérifiée dans un navigateur de la taille d'un téléphone sur un ordinateur, pas sur un vrai téléphone : essaie avant de promettre une salle plus large ; beaucoup plus large, le héros et les objets deviennent petits à voir et à diriger.

Plan de la leçon (90 minutes)

Temps	Partie	Ce qui se passe	Si tu es en retard
0-5	Échauffement	Joue au jeu terminé. Demande : de quelles pièces a-t-il besoin ?	Saute la discussion
5-10	1. Ouvrir	Ouvrir le projet de départ, regarder les ressources, F5	
10-20	2. Héros	Créer un sprite, dessiner le héros	Donne un héros prêt du projet terminé
20-32	3. Objets	Cocher Solide sur le mur, créer 7 objets	Travaillez à deux : l'un lit le tableau, l'autre clique
32-47	4. Labyrinthe	Peindre les murs, placer porte, clé, gemmes, bombes, cœur, but, héros	Donne la salle du projet terminé
47-55	5a. Héros	Sept lignes de blocs	**Priorité 1**
55-59	5b. Gemmes	Du score, puis elles disparaissent	**Priorité 1**
59-63	5c. Clé	Détruire toutes les portes	Priorité 2

63-66	5d. Cœur	Une vie de plus	Priorité 3 : à sauter en premier
66-72	5e. Bombes	Perdre une vie, retour au départ	Priorité 2
72-78	5f. But	La question « reste-t-il des gemmes ? »	**Priorité 1**
78-80	5g. Fin de partie	Autre événement : Plus de vies	Priorité 3 : à sauter en premier
80-87	6. Export	Exporter en HTML5, ouvrir dans le navigateur	Fais-le en démonstration devant la classe
87-90	Conclusion	Échanger et jouer ; les questions « Réfléchis »	

Si le temps manque, passe de 5b directement à 5f : le héros, les gemmes et le but font déjà un jeu complet. Ajoute ensuite la clé, les bombes, le cœur et la fin de partie dans cet ordre.

Déroulement et problèmes courants

Partie 2, le héros. Créer un sprite ouvre une image vide de 32 sur 32. La boîte de collision suit les pixels dessinés (non transparents) : un héros qui remplit le carré est le plus grand, et un héros avec des bords vides se faufile plus facilement. Les deux fonctionnent dans des chemins de deux carrés.

Partie 3, les objets. Huit objets en tout. Seuls le mur et la porte sont Solides.

> dans ce jeu, ce sont les **lignes de collision** qui arrêtent le héros, pas le réglage Solide : un mur Solide tout seul ne l'arrête pas (vérifié : le héros passe à travers). On demande quand même Solide parce que c'est le bon réglage pour les murs et les portes, et que d'autres blocs, comme Rebondir, en dépendent.

Partie 4, le labyrinthe.

- **Deux carrés de large.** Le héros s'arrête là où on relâche la touche, pas sur la grille. Dans un chemin d'un carré, il peut se retrouver décalé de quelques pixels et rester coincé. Les chemins du labyrinthe de référence font deux ou trois carrés de large ; un test le joue avec le héros placé jusqu'à 16 pixels hors de la grille.
- **Une porte** est une colonne d'objets porte en travers d'un chemin, aussi large que le chemin. Les élèves n'en placent souvent qu'une et laissent un trou : le héros la contourne.
- **La clé doit venir en premier.** Elle doit être atteignable sans passer par une porte. Vérifie chaque labyrinthe avant l'échange.
- **Les bombes** se mettent au bord d'un chemin, jamais en travers. Le labyrinthe de référence met chaque bombe sur la rangée du bord d'un chemin de trois carrés.

Partie 5, la programmation.

- 5a, le héros : comme dans « Mon premier labyrinthe ». Sans les deux lignes de **collision**, le héros traverse les murs et les portes.
- 5b, les gemmes : c'est la gemme qui fait le travail : *quand collision avec obj_hero : Ajouter au score 10, Détruire cette instance*.

- 5c, la clé : **Détruire toutes les instances de** obj_door enlève toutes les portes de la salle. Ensuite **Détruire cette instance** enlève la clé.
- 5e, les bombes : c'est le **héros** qui a l'événement, donc **Détruire autre instance** désigne la bombe. Les quatre blocs s'exécutent dans l'ordre : perdre une vie, la bombe disparaît, arrêt, retour au départ. Sans **arrêter le mouvement**, le héros continue de bouger si une touche est maintenue.
- 5f, le but : le bloc **If** demande combien de gemmes existent (*count of object obj_gem == 0*). Les blocs placés dans son emplacement **then** se passent quand la réponse est oui, ceux de son emplacement **else** quand elle est non. C'est la seule idée nouvelle de la leçon : prends ton temps. Le bloc commence avec le nom d'objet obj_coin, donc les élèves doivent écrire obj_gem. Ses mots (If, count of, then, else) restent en anglais même quand l'IDE est en français.
- 5g, fin de partie : le bloc **Autre événement** a une liste déroulante ; choisis **Plus de vies**. **Redémarrer le Jeu** recommence tout : score 0, 3 vies, toutes les gemmes, portes, clés, cœurs et bombes reviennent (vérifié dans le moteur de jeu et dans un navigateur).

Le jeu terminé du projet de référence utilise des messages en anglais (« You win! », « Collect every gem first! », « Game over! ») ; les élèves écrivent les leurs dans leur langue.

Exporter

Fichier > Exporter en HTML5 demande un dossier et propose d'ouvrir le jeu dans le navigateur. Le résultat est **un seul fichier**, nommé d'après le projet. Ce jeu ne contient aucun code Python : il n'a pas besoin d'internet pour jouer. La barre au-dessus du jeu affiche **Score** et **Lives**, que ce jeu utilise vraiment.

Jouer sur un téléphone

Ce qu'on sait :

- La page exportée détecte un écran tactile et affiche alors quatre boutons flèches sous le jeu. Garder un bouton appuyé déplace le héros. Cela a été vérifié dans un navigateur de la taille d'un téléphone, sur un ordinateur, et **pas sur un vrai téléphone**.
- La page du jeu est faite pour les écrans étroits et se réduit pour s'y adapter.
- Chaque mécanisme de ce jeu (gemmes, clé et portes, cœur, bombes, but, fin de partie) a été vérifié dans un navigateur sans fenêtre en plaçant le héros sur chaque objet.

Ce qu'il faut prévoir :

- **Faire arriver le fichier sur le téléphone.** Les téléphones ne peuvent souvent pas ouvrir un fichier .html depuis leur stockage (les iPhone en particulier). La méthode fiable est de mettre le fichier à une **adresse web** (l'espace web de l'école ou un hébergeur gratuit), puis d'ouvrir cette adresse sur le téléphone. Cela n'a pas été testé ici.
- **Vitesse sur un écran rapide.** Le jeu exporté fait un pas à chaque rafraîchissement de l'écran. Sur un écran de 60 Hz, c'est la vitesse prévue. Sur un téléphone avec un écran de 90 ou 120 Hz, le héros ira plus vite, jusqu'à deux fois plus vite. C'est une limite connue de l'export HTML5, pas encore corrigée. Ce labyrinthe reste jouable, mais les bombes sont plus difficiles à éviter à double vitesse.

Différenciation

- **Besoin d'aide :** pars de la salle du projet terminé et ne change que les murs, ou donne un labyrinthe imprimé à recopier. Ne fais que les priorités 1 et 2.
- **Pour aller plus loin :** une deuxième clé et un deuxième groupe de portes, une gemme cachée derrière une porte, ou plus de points pour certaines gemmes. Tout cela utilise des blocs de la boîte à outils Junior.

Évaluer le travail

Demande à chaque élève de te montrer : le héros qu'il a dessiné, un labyrinthe qui peut être terminé, au moins trois mécanismes qui fonctionnent (par exemple les gemmes, une porte avec sa clé, et une bombe), et le fichier .html exporté qui fonctionne dans un navigateur. Les trois questions « Réfléchis » servent de ticket de sortie.

Dépannage

Symptôme	Cause et solution
Le héros traverse les murs ou les portes	Il manque la ligne de collision sur le héros (**Quand collision avec** obj_wall ou obj_door, **arrêter le mouvement**)
Le héros est coincé dans un chemin	Le chemin fait un carré de large. Élargis-le
La clé ne fait rien	**Détruire toutes les instances de** obj_door manque ou vise le mauvais objet
La porte ne peut jamais s'ouvrir	La clé est derrière la porte. Déplace-la
Le but dit toujours « Collect every gem first! »	Une gemme est quelque part d'inatteignable, ou le nombre du bloc **If** n'est pas 0
Le but gagne tout de suite	Le bloc **If** manque, ou son objet n'est pas obj_gem (il commence à obj_coin)
La bombe ne fait rien	L'événement est sur le héros : **Quand collision avec** obj_bomb
Le héros ne peut pas esquiver une bombe	La bombe bloque tout le chemin. Mets-la au bord
Les portes restent après la clé dans le navigateur	Ancienne version de l'export. Mets PyGameMaker à jour et exporte de nouveau
Fenêtre de jeu grise	Ordinateur lent ou à court de mémoire. Attends, puis réessaie
Quelque chose a été supprimé	Outils > Restaurer les ressources supprimées